

Chaînage de prompts et gestion du contexte

Astrid Klipfel

Automne 2025

1 Installation

Documentation à consulter pour ce TP :

- Installation et premiers pas de **uv** : <https://docs.astral.sh/uv/>
- Documentation officielle de **DSPy** : <https://dspy.ai/>
- Guides DSPy (signatures, modules, optimisation) : <https://dspy.ai/learn/>
- API REST d'**ollama** (endpoint `/api/chat`) : <https://github.com/ollama/ollama/blob/main/docs/api.md>
- Client Python officiel d'**ollama** : <https://github.com/ollama/ollama-python>

Astuce Vous pouvez utiliser **uv** pour gérer Python et ses environnements. **uv** est un gestionnaire d'environnements et de dépendances très rapide, écrit en Rust, qui remplace avantageusement **pip**, **pipx** et **virtualenv**. Vous pouvez l'installer avec la commande :

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Une fois installé, initialisez un projet **uv**, ajoutez les dépendances, puis lancez vos scripts ou tests avec `uv run` :

```
# crée un projet uv (pyproject.toml, .env, ...) dans le répertoire courant
uv init
# initialise le dépôt git (committez à chaque tentative/modification)
git init
# ajoute les dépendances au projet
uv add ollama dspy
# exécute un script ou une commande dans l'environnement du projet
uv run python script.py
```

Avec `uv init` et `uv run`, l'environnement virtuel est géré automatiquement : inutile de l'activer.

1.1 DSPy pour gérer les prompts

Plutôt que de construire manuellement les **strings** des prompts, vous pouvez utiliser **DSPy**, un framework qui permet de définir des **signatures** déclaratives et de programmer le chaînage de prompts de façon structurée. Une signature décrit les entrées et la sortie attendue d'un module de prompt. Par exemple, une signature de correction :

```
import dspy
```

```
lm = dspy.LM("ollama/llama3.2:1b")
dspy.configure(lm=lm)
```

```
class CorrectionSignature(dspy.Signature):
    """Corrige l'implémentation d'une fonction à partir du rapport d'erreur."""
```

```

enonce: str = dspy.InputField()
code: str = dspy.InputField()
erreur: str = dspy.InputField()
code_corrige: str = dspy.OutputField()

```

Conseil. DSPy n'est **qu'une suggestion** pour structurer vos prompts, pas une obligation de ce TP. Vous pouvez très bien construire vos prompts à la main avec des **f-strings**. L'essentiel pour l'étape 2 est de **gérer explicitement le contexte** (code précédent, rapport d'erreur) envoyé au modèle, que vous passiez par DSPy ou non.

1.2 API ollama fournie

Un serveur **ollama** est déjà mis à disposition pour ce TP : vous n'avez rien à installer pour le faire fonctionner. Il est accessible à l'adresse `http://172.31.143.202/ollama/` et expose une API REST compatible avec l'endpoint `/api/chat` ; le modèle `qwen3:8b` y est déjà téléchargé.

Conseil. Attention au slash final. À cause de la configuration du **reverse proxy**, l'URL de l'endpoint doit *toujours* se terminer par un `/` : `http://172.31.143.202/ollama/`. Sans ce slash final, le proxy ne réécrit pas le chemin et renvoie une **redirection** (ou une erreur) que DSPy ne suit pas lors d'une requête `POST` – d'où un échec de connexion. C'est d'ailleurs le fix appliqué dans le harnais fourni : conservez systématiquement ce trailing slash dans `api_base`.

DSPy s'y connecte directement via le provider `ollama/` (voir section 1.1). Pour vérifier que la connexion fonctionne, posez une question au modèle en définissant une **signature** :

```

import dspy

lm = dspy.LM("ollama/qwen3:8b", api_base="http://172.31.143.202/ollama/")
dspy.configure(lm=lm)

class ReponseSignature(dspy.Signature):
    """Répond à une question de programmation par du code Python."""
    question: str = dspy.InputField()
    reponse: str = dspy.OutputField()

repondre = dspy.Predict(ReponseSignature)
print(repondre(question="Ecris une fonction add(a, b).").reponse)

```

Le modèle `qwen3:8b` active par défaut un mode de **raisonnement** (*thinking*) : il génère d'abord un raisonnement interne avant de répondre. Cela ralentit les appels et pollue les réponses avec du texte qui n'est pas du code, ce qui gêne l'extraction du code généré (étape 1). Pour ce TP, **désactivez cette option** sur vos appels au modèle en passant le paramètre `think` à `false` dans la requête `/api/chat` :

```

curl http://172.31.143.202/ollama/api/chat -d '{
  "model": "qwen3:8b",
  "think": false,
  "messages": [
    {"role": "user", "content": "Ecris une fonction add(a, b)."}
  ]
}'

```

Documentation de l'API REST d'**ollama** (endpoint `/api/chat`) : <https://github.com/ollama/ollama/blob/main/docs/api.md>

1.3 Installation locale d'ollama (solution de repli)

Si le serveur fourni n'est pas accessible, vous pouvez installer **ollama** localement. **ollama** est un package qui permet de faire tourner un LLM en local sur votre machine. Pour l'installer, utilisez la commande :

```
curl -fsSL https://ollama.com/install.sh | sh
```

Pour tester votre installation en lançant la version 1 milliard de paramètres de llama 3.2. **ollama** va télécharger le modèle et démarrer un shell interactif pour interagir avec le modèle : ceci correspond à l'endpoint `/api/chat` utilisé dans la section précédente.

```
ollama run llama3.2:1b
```

Conseil. C'est volontairement un **modèle** que vous allez utiliser pour ce TP, même si vous pourriez en utiliser un bien plus puissant. L'objectif n'est pas de prendre le modèle le plus puissant disponible, mais d'observer comment un modèle limité peut tout de même résoudre des tâches difficiles *si on structure correctement ses appels*.

2 Objectif du TP

Jusqu'ici, vous avez utilisé un LLM avec un unique prompt, en attendant une réponse immédiatement satisfaisante. Ce TP propose une tâche volontairement **trop difficile** pour un petit modèle local en un seul appel, mais **atteignable** si vous structurez la façon dont vous dialoguez avec lui : enchaînement de prompts, boucle de correction, gestion du contexte envoyé au modèle.

Définition. Un **harnais** vous est fourni : pour chaque tâche d'un petit banc de test (**benchmark**), il demande au LLM d'**implémenter une fonction Python** précisément spécifiée, puis vérifie automatiquement le résultat à l'aide de **tests unitaires** fournis. Vous êtes libre de le modifier pour améliorer vos résultats.

2.1 Le banc de tâches

Le banc de test est déjà fourni et contient **16 tâches**. Chaque tâche est composée de :

- un **énoncé** précis de la fonction à implémenter (signature, comportement attendu, cas limites) ;
- un fichier de **tests unitaires** qui valide automatiquement une implémentation.

Les tâches couvrent des algorithmes classiques (recherche, tris, parcours de chaînes, structures de données) et sont de difficulté croissante : certaines sont volontairement conçues pour mettre un petit modèle en échec dès la première tentative.

2.2 Le harnais fourni

Le harnais (module **evaluator**) est fourni avec le banc. Il expose trois fonctions :

- `get_task_description(task)` : renvoie l'énoncé de la tâche, c'est-à-dire le prompt à envoyer au LLM ;
- `test_implementation(task, code)` : teste une implémentation sur la tâche et renvoie (`valid, message`) où `message` est un rapport d'erreur utile à la correction pour l'étape 2 ;
- `score_implementations(implementations)` : évalue un lot d'implémentations et renvoie le **taux de réussite** et la liste des tâches en échec.

Conseil. Les énoncés, les tests du banc de tâches, ainsi que le **harnais** vous sont fournis. Ce document décrit les stratégies que vous devez mettre en œuvre pour faire progresser le banc.

Les tâches sont choisies pour être difficiles à réussir **du premier coup** avec un petit modèle : erreurs de syntaxe, mauvaise gestion des cas limites, mauvaise interprétation de l'énoncé, etc. L'objectif du TP est de mesurer et d'améliorer le **taux de réussite** sur ce banc à travers des stratégies de plus en plus élaborées.

2.3 Ce que vous allez mesurer

Pour chaque stratégie, vous calculerez sur l'ensemble du banc :

- le **taux de réussite** (nombre de tâches dont les tests passent, sur le total) ;
- le **nombre moyen d'appels au LLM** par tâche ;
- éventuellement, le temps total d'exécution.

Ces mesures vous serviront à comparer objectivement les deux étapes du TP dans votre rapport.

3 Étape 1 – Implémentation directe

Définition. Pour chaque tâche, envoyez l'énoncé au LLM en un **unique prompt**, récupérez le code généré, et exécutez les tests unitaires correspondants. Aucune tentative supplémentaire n'est faite en cas d'échec.

Le harnais fourni sait déjà **tester une implémentation** : à vous d'**extraire le code Python** de la réponse du modèle (attention aux blocs `“python ... “`) avant de l'exécuter. Pour chaque tâche, envoyez l'énoncé au LLM, extrayez le code, puis vérifiez l'implémentation. Pour évaluer l'ensemble du banc en une seule fois, utilisez la fonction `score_implementations` qui teste toutes vos implémentations et renvoie le taux de réussite et la liste des tâches en échec.

Conseil. Cette étape est votre **baseline**. Ne cherchez pas à l'améliorer : c'est volontairement la version la plus simple, elle vous sert de point de comparaison pour la suite.

4 Étape 2 – Boucle de correction avec retour d'erreur

Définition. Reprenez le harnais de l'étape 1, mais en cas d'échec des tests, **réessayez** jusqu'à ce que les tests passent ou qu'un nombre maximum de tentatives soit atteint (par exemple 5). Dans un premier temps, renvoyez simplement le **même prompt initial** (seule la génération change, par variabilité du sampling). Puis, lorsqu'une tentative échoue, **enrichissez le prompt suivant** en y ajoutant le **code précédemment généré** et le **rapport d'erreur** des tests (message d'assertion, traceback) et demandez explicitement au modèle de corriger son implémentation.

- Fixez un budget maximum de tentatives par tâche et respectez-le strictement.
- Le harnais fourni vous renvoie le rapport d'erreur des tests (message d'assertion, traceback) : ne conservez que les informations utiles à la correction (pas tout le bruit de la sortie).
- Construisez un nouveau prompt qui contient : l'énoncé original, la dernière implémentation générée, l'erreur rencontrée, et une instruction de correction.
- Réfléchissez à la **gestion du contexte** sur plusieurs tentatives : faut-il garder tout l'historique des tentatives précédentes, ou seulement la dernière ? Que se passe-t-il si le contexte grandit trop vite (cf. cours sur la compression et l'externalisation du contexte) ?
- Enregistrez, pour chaque tâche, le nombre de tentatives effectuées avant succès (ou l'échec si le budget est épuisé), et distinguez les réussites obtenues *à l'identique* de celles obtenues *après correction*.

Conseil. Commencez par la stratégie "*naïve*" (relancer à l'identique en espérant un tirage plus favorable, sans informer le modèle de ce qui n'allait pas), mesurez-la, puis passez au **chaînage de prompts** : chaque appel au LLM n'est plus indépendant, il exploite le résultat du précédent. C'est là que la correction devient vraiment efficace. Comparez, à budget maximum identique, le nombre de tentatives nécessaires dans les deux variantes.

5 Rendu

Utilisez **git** pour suivre l'évolution de votre travail : **committez après chaque tentative / modification** du harnais avec un message décrivant la stratégie testée et son résultat. Vous enverrez votre dépôt git (le code source complet, historique compris) sur Moodle.

Pour le rendu du TP, vous fournirez :

- votre **harnais** final (le harnais fourni, éventuellement modifié ou complété pour les deux étapes) ;
- un **petit rapport** (markdown ou PDF, 1 à 2 pages) présentant, pour chaque étape, le taux de réussite et le nombre moyen d'appels au LLM sur l'ensemble du banc de tâches.

Le rapport doit surtout détailler **votre exploration** :

- quelles modifications avez-vous apportées au harnais fourni (extraction du code, exécution des tests, gestion du contexte), et pourquoi ?
- comment avez-vous **amélioré vos résultats** entre l'étape 1 et l'étape 2 (pensez à utiliser votre historique git pour justifier chaque changement testé) ? Quel est l'impact mesuré du réessai à l'identique puis du retour d'erreur ? Quels changements de prompt, de stratégie de correction ou de gestion du contexte avez-vous testés, et quels en ont été les effets sur le taux de réussite et le nombre d'appels ?
- une courte discussion : quelles tâches restent en échec même après correction ? Pourquoi, selon vous ? Quelles limites voyez-vous à cette approche de correction automatique ?